Assignment 2

Group Number and candidates:

Group 7
Simen Emil Wiig Holmen
Oda Lunde Opheim
Maja Solberg Petterson
Joel Warraphon Markussen

Course code	IS-309
Course name	Videregående databasesystemer
Emneansvarlig	Rania Fahim Hassan Ibrahim Elgazzar
Due Date	30.04.2025
Title	Assignment 2

We confirm hereby that we are not citing or using in other ways other's work without stating that clearly and that all of the sources that we have used are listed in the references	**Yes** **X**	**No**

We allow the use of this work for educational purposes	**Yes** **X**	**No**

We hereby confirm that every member of the group named on this page has contributed to this deliverable/product	**Yes** **X**	**No**

Table of contents:

Figurelist:

Task 1:

Write the code necessary to implement the specifications.

We used the .tar file to restore the entire database.

Task 2:

Write the scripts that execute the stored procedures and provide a screenshot of the result.

Scripts can be found in https://github.com/simholmen/IS309-assignment2 or Database dump

Create_bicycle_sp

```
call create_bicycle_sp
(NULL,
'classic',
NULL,
NULL,
NULL,
NULL,
NULL,
'available',
40,
190,
10,
12,
NULL)
```

Data Output | Messages | Notifications

```
ERROR:  Invalid Longitude  190.
CONTEXT:  PL/pgSQL function create_bicycle_sp(character varying,integer,character varying,character varying,integer,character varying,integer,integer,numeric,numeric,date) line 31 at RAISE

SQL state: P0001
```

Figure 1: Screenshot of error, when Longitude is not between -180 and 180.

Figure 2: Screenshot of error message, when the status is not from the status table.

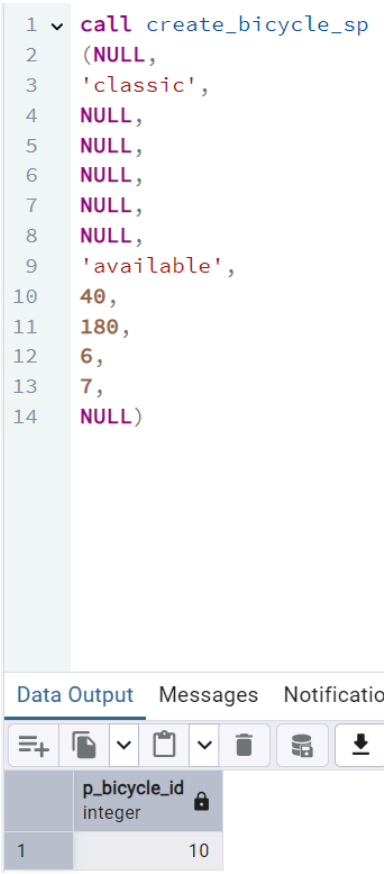

Figure 3: Screenshot of a successful call, where bicycle_id is output

Purchase_membership_sp

```
CALL PURCHASE_MEMBERSHIP_SP (
    NULL,
    'Heartland Monthly Pass',
    20,
    '2025-04-04',
    '2025-06-30',
    11
);
```

Data Output | Messages | Notifications

	p_membership_id integer
1	3

Figure 4: Screenshot of a successful call, where membership_id is output.

```
CALL PURCHASE_MEMBERSHIP_SP (
    NULL,
    'Heartland Monthly Pass',
    20,
    '2025-04-04',
    '2025-06-30',
    999
);
```

Data Output | Messages | Notifications

```
ERROR:  invalid account (999).
CONTEXT:  PL/pgSQL function purchase_membership_sp

SQL state: P0001
```

Figure 5: Screenshot of an error message for invalid account id

```
CALL PURCHASE_MEMBERSHIP_SP (
    NULL,
    'Heartland Monthly Pass',
    20,
    '2025-04-04',
    NULL,
    11
);
```

Data Output | Messages | Notifications

```
ERROR:  Missing mandatory value for parameter p_end_time in PURCHASE_MEMBERSHIP_SP.  No membership added.
CONTEXT:  PL/pgSQL function purchase_membership_sp(character varying,numeric,date,date,integer) line 16 at

SQL state: P0001
```

Figure 6: Screenshot of error message for missing value for end time

Add_dock_sp

Query | Query History

```
CALL add_dock_sp('bcycle_heartland_1917', 26, 'occupied', 1003)
```

Data Output | Messages | Notifications

```
CALL

Query returned successfully in 60 msec.
```

Figure 7: Add_dock_sp was called successfully

Query | Query History

```
CALL add_dock_sp('Worng bycycle', 26, 'occupied', 1003)
```

Data Output | Messages | Notifications

```
ERROR:  Invalid station identifier
CONTEXT:  PL/pgSQL function add_dock_sp(character varying,integer,character varying,integer) line 14 at RAISE

SQL state: P0001
```

Figure 8: Wrong station_id

Query Query History

```
1  CALL add_dock_sp('bcycle_heartland_1917', 2, 'occupied', 1003)
```

Data Output Messages Notifications

```
ERROR:  Dock number 2 already exists.
CONTEXT:  PL/pgSQL function add_dock_sp(character varying,integer,character varying,integer) line 27 at RAISE

SQL state: P0001
```

Figure 9: Already existing dock_id

Create_station_sp

Query Query History

```
1  CALL CREATE_STATION_SP(
2      'test6','Central Station','C123',59.9139,10.7522,'bergen Central','0150','12345678'
3      ,50,10,40,1,1,CURRENT_DATE,'bergen-city-bike'
4  );
5
```

Data Output Messages Notifications

```
CALL

Query returned successfully in 42 msec.
```

Figure 10: Create_station_sp was called successfully

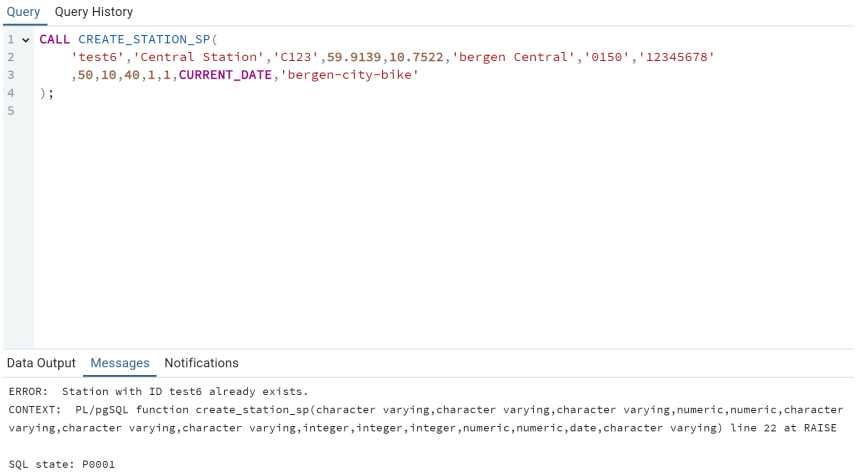

Figure 11: Create_station_sp was called unsuccessful due to multiple stations with the same ID

Figure 12: Create_station_sp was called unsuccessfully due to values being null.

Create_account_sp

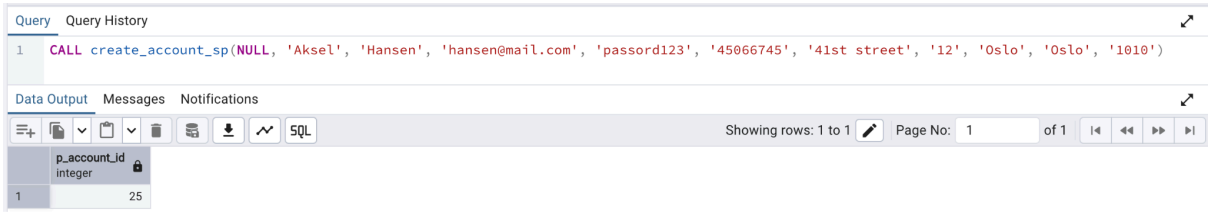

Figure 13: create_account_sp was called successfully, and returned with the created account_id

Figure 14: Error message when email already exists

Figure 15: Error message when a not nullable input is NULL

Task 3:

Create one stored function, write also the SQL script that executes the stored function and provide a screenshot of the result (justify the purpose of the stored function).

We created a stored function which adjusts the prices of bc_pass with 10%. The reasoning behind this function is to be able to easily raise the prices of the passes to account for inflation. We also rounded the number to only include 3 decimals to make it easier to view.

Adjust_for_inflation

Query | Query History

```
1  CREATE OR REPLACE FUNCTION ADJUST_FOR_INFLATIONN()
2  RETURNS TABLE (adjusted_cost NUMERIC) AS $$
3
4  BEGIN
5      UPDATE bc_pass
6      SET pass_cost = ROUND(pass_cost * 1.10, 3);
7
8      RETURN QUERY
9      SELECT pass_cost FROM bc_pass;
10
11 END;
12 $$ LANGUAGE plpgsql;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 52 msec.
```

Figure 16: Funksjon adjust_for_inflation

Query | Query History

```
1  SELECT adjust_for_inflation();
```

Data Output | Messages | Notifications

	adjust_for_inflation numeric
1	13.200
2	22.000
3	171.600

Figure 17: Pass_cost after using adjust_for_inflation

Task 4:

Create two triggers; one at the row level and one at the statements level, and describe the purpose of the two triggers and their logic (write the PL/SQL scripts that create them).

Row level trigger:

```
CREATE TABLE inflasjon_update (
    pass_id INTEGER,
    pass_cost NUMERIC,
    new_pass_cost NUMERIC,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Opprett trigger-funksjonen for logging
CREATE OR REPLACE FUNCTION log_update_inflation()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO inflasjon_update (
        pass_id,
        pass_cost,
        new_pass_cost,
        updated_at)
    VALUES (
        OLD.pass_id,
        OLD.pass_cost,
        NEW.pass_cost,
        NOW()
    );
    RAISE NOTICE 'Updated pass_id % with new cost %', OLD.pass_id, NEW.pass_cost;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
-- Opprett triggeren som utløses etter oppdatering av pass_cost
CREATE OR REPLACE TRIGGER update_inflasjon
AFTER UPDATE OF pass_cost ON bc_pass
FOR EACH ROW
EXECUTE FUNCTION log_update_inflation();
```

Figure 18: Row level trigger

The logic behind implementing this row level trigger is to automatically log changes to the pass_cost row of the bc_pass table. The result of this is whenever the pass_cost row is updated, the trigger stores both the old and the new value in the table inflasjon_update. In addition to this, the trigger secures a timestamp of the changes made through the updated_at ROW, securing control over the changes to the database. The trigger is designed in order to ensure a historical overview of the changes to the price, which can be useful in order to for example analyse the price history in the future.

Statement level trigger:

```
-- Opprett trigger-funksjonen
CREATE OR REPLACE FUNCTION log_membership_update()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'New update on table % at % with operation %', TG_TABLE_NAME, current_timestamp, TG_OP;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

-- Opprett trigger som utløsen når noen gjør endiringer på bc_membership
CREATE OR REPLACE TRIGGER trigger_log_membership_update
AFTER INSERT OR UPDATE OR DELETE ON bc_membership
FOR EACH STATEMENT
EXECUTE FUNCTION log_membership_update();
```

Figure 19: Statement level trigger

A statement trigger, unlike the row level trigger, is better suited for tasks on a higher-level. While the row-level triggers focus on specific rows and triggers whenever a row is updated, the statement level trigger only fires once for an entire SQL statement, no matter how many rows are impacted.

The function of this trigger is called log_membership_update(). It states that whenever an operation is performed on the bc_membership table, the function is triggered for the whole SQL statement. The function is using the RAISE NOTICE feature to generate a message in the log, containing important information such as the name of the affected table, the timestamp for the operation and the type of operation. The trigger fires once the connection function is activated.

The logic behind the implementation of this trigger is that it gives insight into the changes that are applied to the bc_membership table, without requiring the logging of individual rows. This is ideal for monitoring the activity of the table on an overarching level (PiEmbSysTech, 2025). Different statistics can also be retrieved by monitoring the activity of the membership table. These statistics could be used for marketing purposes, by checking how often a membership is renewed.

Task 5:

Provide a participation sheet specifying the contribution of each group member.

Participation sheet:

Name	Joel	Maja	Oda	Simen
Procedures:				
create_account_sp	x	x	x	x
create_bicycle_sp			x	
create_station_sp	x			
add_dock_sp				x
purchase_membership_sp		x		
Functions:				
adjust_for_inflation	x	x	x	x
Tiggers:				
log_membership_update	x	x	x	x
log_update_inflation	x	x	x	x

Figure 20: Participation sheet

Task 6:

Given the tables created for Bcycle database: BC_ACCOUNT, BC_BICYCLE, BC_BICYCLE_STATUS, BC_COUNTRY, BC_DOCK, BC_MEMBERSHIP, BC_PASS, BC_PROGRAM, BC_STATION, and BC_TRIP.

1. **Provide an overview of the Bcycle database physical storage structure, database size, size of each relation, and how many pages each tuple is stored in.**

Physical storage

```
SELECT pg_size_pretty(pg_database_size('Bcycle'));
```

Data Output | Messages | Notifications

Showing rows: 1

	pg_size_pretty text
1	8083 kB

Figure 21: Size of the whole database

```
SELECT relname AS "Table",
       pg_size_pretty(pg_total_relation_size(relid)) AS "Total Size",
       pg_size_pretty(pg_relation_size(relid)) AS "Table Size",
       pg_size_pretty(pg_indexes_size(relid)) AS "Indexes Size"
FROM pg_catalog.pg_statio_user_tables;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 Page No: 1

	Table name	Total Size text	Table Size text	Indexes Size text
1	bc_program	24 kB	8192 bytes	16 kB
2	bc_country	56 kB	16 kB	16 kB
3	bc_trip	32 kB	8192 bytes	16 kB
4	bc_bicycle	24 kB	8192 bytes	16 kB
5	bc_bicycle_status	56 kB	32 kB	0 bytes
6	bc_dock	24 kB	8192 bytes	16 kB
7	bc_station	24 kB	8192 bytes	16 kB
8	bc_account	40 kB	8192 bytes	32 kB
9	bc_membership	32 kB	8192 bytes	16 kB
10	bc_pass	32 kB	8192 bytes	16 kB

Figure 22: Total size of each individual table, size of only the table, and size of indexes for each table.

```sql
SELECT relname, oid, relpages, reltuples
FROM pg_class
WHERE relname IN ('bc_account', 'bc_bicycle', 'bc_bicycle_status', 'bc_country',
                  'bc_dock', 'bc_membership', 'bc_pass', 'bc_program',
                  'bc_station', 'bc_trip');
```

Figure 23: query to show relpages and reltuples for each table

	relname name	oid [PK] oid	relpages integer	reltuples real
1	bc_country	16939	2	249
2	bc_bicycle	16924	1	11
3	bc_membership	16947	1	6
4	bc_account	16920	1	4
5	bc_bicycle_status	16933	4	420
6	bc_dock	16942	1	24
7	bc_pass	16953	1	3
8	bc_program	16959	1	2
9	bc_station	16962	1	10
10	bc_trip	16967	1	5

Figure 24: overview of relpages and reltuples for each table

relpages is the number of pages allocated to the relation in the database.

reltuples is the number of tuples(rows) within the relation.

Query Query History

```sql
SELECT 'bc_account' AS table_name, *
FROM pgstattuple('bc_account')
UNION ALL

SELECT 'bc_bicycle' AS table_name, *
FROM pgstattuple('bc_bicycle')
UNION ALL

SELECT 'bc_bicycle_status' AS table_name, *
FROM pgstattuple('bc_bicycle_status')
UNION ALL

SELECT 'bc_country' AS table_name, *
FROM pgstattuple('bc_country')
UNION ALL

SELECT 'bc_dock' AS table_name, *
FROM pgstattuple('bc_dock')
UNION ALL

SELECT 'bc_membership' AS table_name, *
FROM pgstattuple('bc_membership')
UNION ALL

SELECT 'bc_pass' AS table_name, *
FROM pgstattuple('bc_pass')
UNION ALL

SELECT 'bc_program' AS table_name, *
FROM pgstattuple('bc_program')
UNION ALL

SELECT 'bc_station' AS table_name, *
FROM pgstattuple('bc_station')
UNION ALL

SELECT 'bc_trip' AS table_name, *
FROM pgstattuple('bc_trip');
```

Figure 25: Query to get an overview of tuples

	table_name text	table_len bigint	tuple_count bigint	tuple_len bigint	tuple_percent double precision	dead_tuple_count bigint	dead_tuple_len bigint	dead_tuple_percent double precision	free_space bigint	free_percent double precision
1	bc_account	8192	4	512	6.25	0	0	0	7628	93.12
2	bc_bicycle	8192	11	1256	15.33	0	0	0	6824	83.3
3	bc_bicycle_status	32768	420	29096	88.79	0	0	0	704	2.15
4	bc_country	16384	249	12281	74.96	0	0	0	2028	12.38
5	bc_dock	8192	24	1575	19.23	0	0	0	6428	78.47
6	bc_membership	8192	6	312	3.81	0	0	0	7804	95.26
7	bc_pass	8192	3	640	7.81	0	0	0	7504	91.6
8	bc_program	8192	2	321	3.92	0	0	0	7828	95.56
9	bc_station	8192	10	1718	20.97	0	0	0	6356	77.59
10	bc_trip	8192	5	520	6.35	0	0	0	7624	93.07

Figure 26: More detailed overview of how the storage is used in each table

Table_len is the total size of the table on the disk measured in bytes. 8KB equals one page in PostgreSQL.

Tuple_count is how many live tuples(rows) there are. Live tuples are rows that currently are being used and not marked for deletion.

Tuple_len is the combined size of all the tuples in the table, measured in bytes.

Tuple_percent tells us how many percent of the table size is occupied by live tuples. The rest of the space is taken up by dead tuples, free space or other table overhead.

Dead_tuple_count is how many dead tuples there are in the database.

Dead_tuple_len is the total size of dead tuples in the database, measured in bytes.

Dead_tuple_percent tells us how many percent of the table size is taken up by dead tuples. Dead tuples take up space, but do not contribute to active data.

Free_space is the amount of free space in the table, measured in bytes.

Free_percent is the percentage of the table that is currently free space.

2. **Provide a data dictionary view of the Bcycle database.**

Data dictionary

Data dictionary serves as documentation for the database structure. A data dictionary gives information about the tables and their columns. It includes data type, key constraints, nullability and description. The metadata that is included in a dictionary can assist in defining the scope, characteristics and usage for data elements (UC Merced Library, n.d.).

To view the Data Dictionary the group used a SQL script from The Data Dictionary Dump (DataResearchLabs, 2022).

	schema_nm name	table_nm name	obj_typ text	ord integer	is_key text	column_nm name	data_typ text	nullable text	column_descr text
1	public	Account	TBL	1	PK	accountId	character varying(256)	NOT NULL	[null]
2	public	Account	TBL	2		email	character varying(256)	NOT NULL	[null]
3	public	Account	TBL	3		firstName	character varying(256)	NOT NULL	[null]
4	public	Account	TBL	4		lastName	character varying(256)	NOT NULL	[null]
5	public	Account	TBL	5		password	character varying(256)	NOT NULL	[null]
6	public	Account	TBL	6		phoneNumber	character varying(15)	NOT NULL	[null]
7	public	Account	TBL	7	FK	membershipID	integer(32)	NULL	[null]
8	public	Bycicle	TBL	1	PK	bycicleId	character varying(30)	NOT NULL	[null]
9	public	Bycicle	TBL	2	FK	bycicleTypeId	integer(32)	NOT NULL	[null]
10	public	Bycicle	TBL	3		make	character varying(100)	NOT NULL	[null]
11	public	Bycicle	TBL	4		model	character varying(100)	NOT NULL	[null]
12	public	Bycicle	TBL	5		color	character varying(30)	NOT NULL	[null]
13	public	Bycicle	TBL	6		yearAquired	smallint(16)	NOT NULL	[null]
14	public	Bycicle	TBL	7		bikeStatus	character varying(15)	NOT NULL	[null]
15	public	Bycicle	TBL	8		batteryStatus	smallint(16)	NULL	[null]
16	public	Bycicle	TBL	9		bikeLong	character varying(11)	NOT NULL	[null]
17	public	Bycicle	TBL	10		bikeLat	character varying(12)	NOT NULL	[null]
18	public	BycicleType	TBL	1	PK	bycicleTypeId	integer(32)	NOT NULL	[null]
19	public	BycicleType	TBL	2		bycicleTypeName	character varying(256)	NOT NULL	[null]
20	public	BycicleType	TBL	3		color	character varying(50)	NOT NULL	[null]
21	public	BycicleType	TBL	4		make	character varying(256)	NOT NULL	[null]
22	public	BycicleType	TBL	5		model	character varying(256)	NOT NULL	[null]
23	public	Dock	TBL	1	PK	dockId	integer(32)	NOT NULL	[null]
24	public	Dock	TBL	2	FK	bycicleId	character varying(30)	NULL	[null]
25	public	Dock	TBL	3		isOccupied	boolean	NOT NULL	[null]
26	public	Membership	TBL	1	PK	membershipId	integer(32)	NOT NULL	[null]
27	public	Membership	TBL	2	FK	membershipTypeNa...	character varying(256)	NOT NULL	[null]
28	public	Membership	TBL	3		purchaseDate	date(3)	NOT NULL	[null]
29	public	Membership	TBL	4		expirationDate	date(3)	NOT NULL	[null]
30	public	MembershipType	TBL	1	PK	membershipTypeNa...	character varying(256)	NOT NULL	[null]
31	public	MembershipType	TBL	2		membershipTypePri...	character varying(256)	NOT NULL	[null]
32	public	Programs	TBL	1	PK	programId	integer(32)	NOT NULL	[null]
33	public	Programs	TBL	2		country	character varying(3)	NOT NULL	[null]
34	public	Programs	TBL	3		location	character varying(100)	NOT NULL	[null]
35	public	Programs	TBL	4		programName	character varying(100)	NOT NULL	[null]
36	public	Programs	TBL	5		phoneNumber	character varying(15)	NOT NULL	[null]
37	public	Programs	TBL	6		programEmailAddress	character varying(256)	NOT NULL	[null]
38	public	Programs	TBL	7		timeZone	character varying(10)	NOT NULL	[null]
39	public	Programs	TBL	8		programURL	character varying(256)	NOT NULL	[null]
40	public	Station	TBL	1	PK	stationId	character varying(30)	NOT NULL	[null]
41	public	Station	TBL	2	FK	statusId	integer(32)	NULL	[null]
42	public	Station	TBL	3		stationAddress	character varying(100)	NOT NULL	[null]
43	public	Station	TBL	4		stationName	character varying(100)	NOT NULL	[null]
44	public	Station	TBL	5		stationLat	character varying(11)	NOT NULL	[null]
45	public	Station	TBL	6		stationLong	character varying(12)	NOT NULL	[null]

46	public	Station	TBL	7		stationPostal	character varying(10)	NOT NULL	[null]
47	public	Station	TBL	8		stationPhone	character varying(15)	NOT NULL	[null]
48	public	Station	TBL	9		totalDocks	character varying(3)	NOT NULL	[null]
49	public	Station	TBL	10	FK	programId	integer(32)	NOT NULL	[null]
50	public	StationStatus	TBL	1	PK	statusId	integer(32)	NOT NULL	[null]
51	public	StationStatus	TBL	2	FK	dockId	integer(32)	NOT NULL	[null]
52	public	StationStatus	TBL	3		totalBikes	character varying(3)	NOT NULL	[null]
53	public	TripData	TBL	1	PK	tripDataId	integer(32)	NOT NULL	[null]
54	public	TripData	TBL	2	FK	bycicleId	character varying(30)	NOT NULL	[null]
55	public	TripData	TBL	3	FK	accountId	character varying(256)	NOT NULL	[null]
56	public	TripData	TBL	4	FK	startingStation	character varying(256)	NOT NULL	[null]
57	public	TripData	TBL	5	FK	endingStation	character varying(256)	NOT NULL	[null]
58	public	TripData	TBL	6		startTime	timestamp with time zone(6)	NOT NULL	[null]
59	public	TripData	TBL	7		endTime	timestamp with time zone(6)	NOT NULL	[null]
60	public	TripData	TBL	8		totalTime	character varying(256)	NOT NULL	[null]
61	public	TripData	TBL	9		totalDistance	integer(32)	NOT NULL	[null]
62	public	TripData	TBL	10		totalCost	character varying(256)	NOT NULL	[null]

Figure 27: Data dictionary for the bcycle database

3. **Discuss the impact of data types and indexes on storage space and effectiveness of query performance with example from the Bcycle database**

Datatypes

Datatypes are essential in relational databases since they determine how data is stored, managed and interacted with. Choosing the right datatype ensures data integrity, efficient storage, query performance and application compatibility (GeeksForGeeks, 2025b).

Numeric Data Types

INT and NUMERIC are exact data types that are usually used when precise numbers are needed, like quantities and costs. INT stores whole numbers while NUMERIC allows for decimals. FLOAT and REAL are also numeric data types, but these are types that are more commonly used for scientific/large datasets that demand less precision. The advantage of these datatypes is that they take up less space. FLOAT and REAL are not appropriate for this context, since the database handles precise, smaller financial data. INT is used for bicycle_rider_capacity and bicycle_year_acuired since these are data that don't require decimals. For bicycle_longitude and bicycle_latitude NUMERIC is used, since these are data that require decimals. NUMERIC often takes up a little more space than INT but allows for more precise numbers. (GeeksForGeeks, 2025b)

Character and String Data Types

VARCHAR is efficient when it comes to storage, as it takes up only the space for the actual string length. CHAR takes up a fixed space. Most of the text or character-based data use VARCHAR as the datatype, except for country_code in the bc_country table. Since country codes always are 2-3 characters, this is a valid choice of datatype. CHAR would not be appropriate for other text fields like account_first_name and account_last_name in the bc_account table, since the input length will vary from user to user, and having a fixed length most likely would take up more space than needed. (GeeksForGeeks, 2025b)

Date and Time Data Type

All the date and time related data have the DATE data type. This data type stores only the calendar date: year, month day. TIMESTAMP is also a data type that stores date related data, but includes the time: hour, minute and second. DATE is a decent choice of data type when the time is irrelevant, which is correct for most of the situations. However, trip_start_time and trip_end_time in the bc_trip table use DATE, but realistically, a trip does not last over a day. Most likely a trip lasts for minutes, maybe hours. Therefore, TIMESTAMP would be a more suitable data type for this situation. TIMESTAMP does take up more storage than DATE, but in return offers more precise logging of the data. (GeeksForGeeks, 2025b)

Indexing

Some of the advantages of indexing include improved query performance, efficient data access, optimized data storing consistent data performance and enforced data integrity. There can, however, be disadvantages with indexing. For example, indexes take up storage space, increase database maintenance overhead and can reduce the insert and update performance (GeeksForGeeks, 2023).

The B-tree index is one of the most common methods for indexing in databases and is designed to optimize search and retrieval processes. B-trees are an efficient and smart way to handle large amounts of data. They make searching, adding and deleting fast and reliable, due to their balanced structure and ability to store multiple keys in one node. However, b-trees are disk-based data structures that may have high disk usage. For smaller datasets, a binary

search may be faster than a b-tree, since each node may contain multiple keys (GeeksForGeeks, 2025a).

B-tree index is used for all the indexes in the Bcycle database. Every index is unique and b-tree indexing ensures efficient lookups, is suitable for range queries and ensures maintaining ordered data (Dicken, 2024).

	indexname name	indexdef text
1	account_email_un	CREATE UNIQUE INDEX account_email_un ON public.bc_account USING btree (account_email)
2	account_pk	CREATE UNIQUE INDEX account_pk ON public.bc_account USING btree (account_id)
3	bicycle_pk	CREATE UNIQUE INDEX bicycle_pk ON public.bc_bicycle USING btree (bicycle_id)
4	country_pk	CREATE UNIQUE INDEX country_pk ON public.bc_country USING btree (country_code)
5	dock_pk	CREATE UNIQUE INDEX dock_pk ON public.bc_dock USING btree (dock_id, station_id)
6	bc_membership_pkey	CREATE UNIQUE INDEX bc_membership_pkey ON public.bc_membership USING btree (membership…
7	program_pk	CREATE UNIQUE INDEX program_pk ON public.bc_program USING btree (program_id)
8	bc_pass_pkey	CREATE UNIQUE INDEX bc_pass_pkey ON public.bc_pass USING btree (pass_id)
9	station_id_unique	CREATE UNIQUE INDEX station_id_unique ON public.bc_station USING btree (station_id)
10	trip_pk	CREATE UNIQUE INDEX trip_pk ON public.bc_trip USING btree (trip_id)

Figure 28: Index name and definition for the different tables

Efficiency for tables with and without index

A sequential scan means that PostGreSQL goes through every row in a given table to find the relevant data. An index scan uses an index to locate the rows that match the query criteria. Index scans are generally more efficient, but there are scenarios where sequential scans are more efficient. For example, sequential scans are more efficient for queries that return a high percentage of rows from the table. PostgreSQL's query planner evaluates the different execution plans and chooses the most efficient one based on the specific query and data distribution. By using the EXPLAIN ANALYZE command we get an in-depth analysis that returns actual execution times along with the query plan. It also provides information about the computational cost of scanning the table (restack, 2025).

```
1  EXPLAIN ANALYZE
2  SELECT station_id, station_capacity, station_address
3  FROM bc_station
4  WHERE station_capacity BETWEEN 5 AND 15
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	QUERY PLAN text
1	Seq Scan on bc_station (cost=0.00..1.15 rows=1 width=190) (actual time=0.021..0.024 rows=6 loops=...
2	Filter: ((station_capacity >= 5) AND (station_capacity <= 15))
3	Rows Removed by Filter: 4
4	Planning Time: 0.089 ms
5	Execution Time: 0.037 ms

Figure 29: Analyze for query on table with index

```
1  ALTER TABLE bc_station DROP CONSTRAINT station_id_unique CASCADE
2
3
4
```

Data Output | Messages | Notifications

```
NOTICE:  drop cascades to 3 other objects
ALTER TABLE

Query returned successfully in 71 msec.
```

Figure 30: Removing primary key and index

```
1  EXPLAIN ANALYZE
2  SELECT station_id, station_capacity, station_address
3  FROM bc_station
4  WHERE station_capacity BETWEEN 5 AND 15
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

	QUERY PLAN text
1	Seq Scan on bc_station (cost=0.00..1.15 rows=1 width=190) (actual time=0.059..0.066 rows=6 loops=...
2	Filter: ((station_capacity >= 5) AND (station_capacity <= 15))
3	Rows Removed by Filter: 4
4	Planning Time: 1.328 ms
5	Execution Time: 0.119 ms

Figure 31: Analyse of query on table without index

Since this database is relatively small, whether the table is indexed or not will not have that much effect on the query efficiency. Postgre chooses to do a sequential scan for both the table with and without index. The computational cost is very low for both queries, since the dataset is very small. For the first query the planning time was longer than the second query. This indicates that PostgreSQL used more time to determine if it should use an index scan or a sequential scan. However, the actual query was very fast in both scenarios. For the query on the table without index, there was no need to determine which type of scan PostgreSQL should use, and therefore the planning time was shorter.

Task 7:

Create the following roles:

1. **The first has the least possible privileges, to only read your tables. It should not have the privileges to create anything in the database.**

Figure 32: Read only on all tables in the database to bicyle_viewer.

2. **The second shall have the privilege to execute all of the procedures and functions created in tasks 1 and 3. This would be a Bcycle administrative position.**

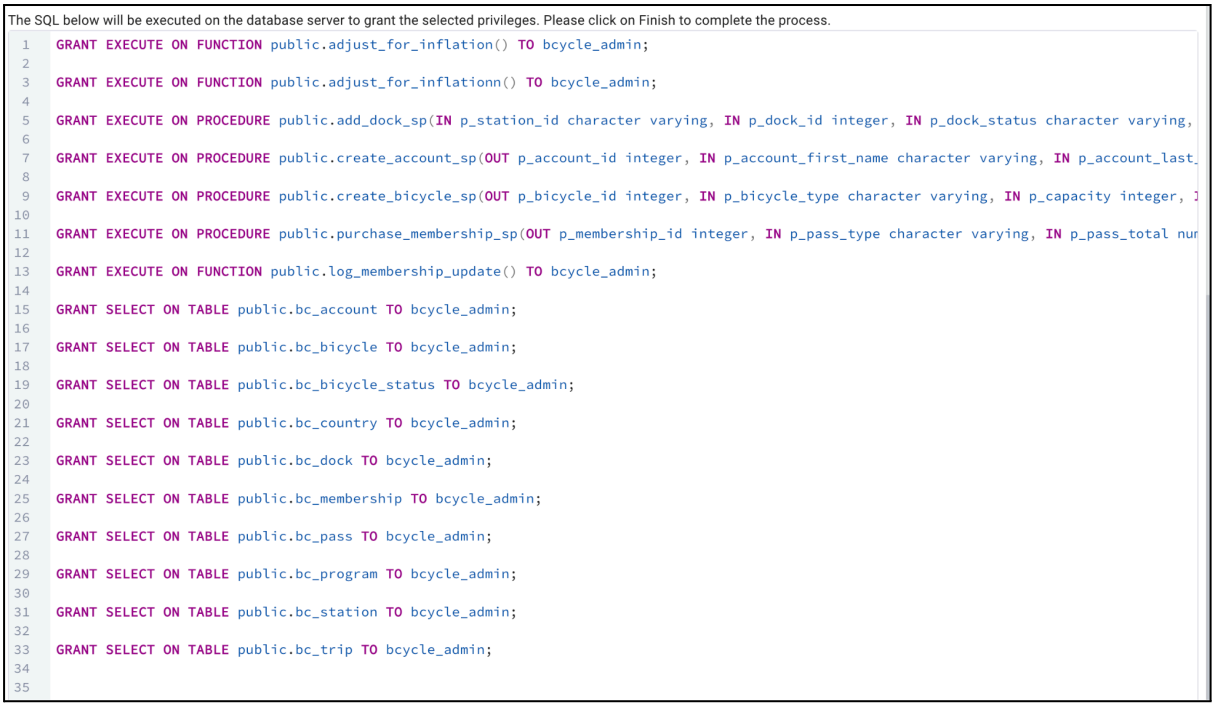

Figure 33: An administrative use with execute privilege for the procedures and functions

3. **The third should have the privilege to execute the procedures to create a new account, a new membership, and a new trip. This would be an account administrator role.**

```
GRANT EXECUTE ON PROCEDURE public.create_account_sp(OUT p_account_id int

GRANT EXECUTE ON PROCEDURE public.purchase_membership_sp(OUT p_membershi

GRANT INSERT ON TABLE public.bc_trip TO bcycle_account_administrator;
```

Figure 34: An account administrator with execute privilege to create an account, create a membership and insert in bc_trip

4. **What difficulties did you encounter in (3) in comparison to (1) and (2)? if so, then how can this problem be solved?**

A problem that we met when creating the account administrator, is that we did not have a procedure which created a new trip. However we solved that problem by granting the account administrator the privilege of inserting to bc_trip and executing on the other procedures.

Task 8:

Create at least other individual and group roles you find important to have for Bcycle database system. Justify the need for the roles you have created.

1. **Grant different privileges on the database schema to the roles you have created.**

```
GRANT EXECUTE ON PROCEDURE public.purchase_membership_sp(OUT p_membership_id integer, IN p_pass_type ch

GRANT EXECUTE ON FUNCTION public.log_membership_update() TO "Financial_administrator";

GRANT EXECUTE ON FUNCTION public.log_update_inflation() TO "Financial_administrator";

GRANT INSERT, SELECT, UPDATE ON TABLE public.bc_membership TO "Financial_administrator";

GRANT INSERT, SELECT, UPDATE ON TABLE public.bc_pass TO "Financial_administrator";

GRANT INSERT, SELECT, UPDATE ON TABLE public.inflasjon_update TO "Financial_administrator";
```

Figure 35: Creating finance_manager with insert, select, update and execute

A financial manager needs to read data from the finance-related tables to be able to create financial reports and analyzes. They need to be able to execute functions like logging of

memberships and updating inflation. The financial manager should alså be able to add new financial records to the database, and update financial data like modifying prices etc.

```
GRANT EXECUTE ON PROCEDURE public.create_bicycle_sp(INOUT p_bicycle_id integer, IN p_bicycle_type character varyin

GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE public.bc_bicycle TO bicycle_operator;

GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE public.bc_bicycle_status TO bicycle_operator;

GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE public.bc_dock TO bicycle_operator;

GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE public.bc_program TO bicycle_operator;

GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE public.bc_station TO bicycle_operator;

GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE public.bc_trip TO bicycle_operator;

```

Figure 36: creating bicycle_operator with insert, select, update and delete

A bicycle operator should be able to view all information about bicycles. They should also be able to add new bicycles, and update existing records like updating the status of the bike, marking it as repaired or in use. They should be able to remove records too like retiring a bike from the system or deleting duplicate entries.

Task 9:

One privilege that we would argue that could be granted to all of the users of the Bcycle database system would be the SELECT privilege. When granting privilege it is important to consider various different factors that could impact the integrity of the database system, such as security, protecting the data and performance. After discussing the concept of privilege of the group, it was agreed upon that the least possible amount of privilege granted makes for the safest approach. This will ensure that people accessing the database only have the authorization to interact with parts of the database that concern them. This decreases the likelihood that something will go wrong, due to users accessing data they are not supposed to.

The reason we chose the SELECT privilege is that we feel like it is the lowest tier of permission you are able to grant to a user, and therefore a good starting point to go off of if every user is to be given access. The SELECT privilege allows the user to retrieve and view data from the tables in the database, but it does not allow for changes. In practice, this means that users with this privilege can not INSERT, UPDATE or DELETE tables, limiting the risk of someone making a mistake as none of the tables or its contents can be changed (PostgreSQL Global Development Group, 2005).

Sources

Dicken, B. (2024, November 9.). *What is a B-tree.* PlanetScale.

https://planetscale.com/blog/btrees-and-database-indexes

DataResearchLabs. (2022, April 23.). *Data Dictionary dump.*

raw.githubusercontent.com/DataResearchLabs/sql_scripts/main/postgresql/data_dictionary/data_dict_dump.sql

GeeksForGeeks. (2023, November 7.). *Indexing in Databases – Set 1*.

Indexing in Databases – Set 1 | GeeksforGeeks

GeeksForGeeks. (2025a, January 29). *Introduction of B-Tree*.

Introduction of B-Tree | GeeksforGeeks

GeeksForGeeks. (2025b, January 30). *SQL Data Types*.

SQL Data Types | GeeksforGeeks

PiEmbSysTech. (2025, March 7). Understanding statement-level triggers in PL/pgSQL: A complete guide.

Understanding Statement-Level Triggers in PL/pgSQL

PostgreSQL Global Development Group. (2005). *5.7 Privileges*

Privileges PostgresQL

Restack. (2025, April 22). *Seq Scan Vs Index Scan In Postgres*.

Seq Scan Vs Index Scan In Postgres | Restackio

UC Merced Library (n.d.) *What Is a Data Dictionary?*

https://library.ucmerced.edu/data-dictionaries